

AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 7: Eigensystems

Adam Prugel-Bennett

September 14, 2026

1 Keywords

- Eigenvectors, Orthogonal Matrices, Eigenvector Decomposition

2 Main Points

2.1 Eigen-Systems

- We can understand ill-conditioning for linear regression by the eigen-decomposition of $\mathbf{M} = \mathbf{X}^T \mathbf{X}$
- This should be revision
- You know that an *eigenvector*, $\mathbf{v}$, satisfies $\mathbf{M} \mathbf{v} = \lambda \mathbf{v}$
- For a symmetric matrix there are n real orthogonal eigenvectors
- You can prove they are orthogonal
- **Orthogonal Matrices**
 - Putting the n eigenvectors into a matrix $\mathbf{V}$ with columns $\mathbf{v}_i$ we obtain an orthogonal matrix
 - The defining property of an orthogonal matrix is $\mathbf{V}^T \mathbf{V} = \mathbf{V} \mathbf{V}^T = \mathbf{I}$
 - They correspond to rotations (with a possible reflection)
- **Matrix Decomposition**
 - We can decompose a symmetric matrix, $\mathbf{M}$ as

$$\mathbf{M} = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^T$$

- Where $\mathbf{\Lambda}$ is a diagonal matrix of eigenvalues of $\mathbf{M}$ (i.e. $\Lambda_{ii} = \lambda_i$)
- $\mathbf{V}$ is the orthogonal matrix made up of the eigenvectors of $\mathbf{M}$
- We can interpret the mapping of a symmetric matrix $\mathbf{M}$ as equivalent to
 1. a rotation defined by $\mathbf{V}^T$
 2. scaling of the i^{th} component by λ_i and
 3. a rotation backwards given by $\mathbf{V}$
- Equivalently if we work in a coordinate system defined by the eigenvectors of $\mathbf{V}$ (this forms an orthonormal basis set) then we just rescale in the directions $\mathbf{v}_i$ by λ_i
 - A symmetric matrix just squashes or expands in different orthogonal directions (this is what the eigensystem captures)

- **Inverse Matrices**

- The inverse of a symmetric matrix $\mathbf{M}$ is given by

$$\mathbf{M} = \mathbf{V} \mathbf{\Lambda}^{-1} \mathbf{V}^T$$

- Where $\mathbf{\Lambda}^{-1}$ is a diagonal matrix with elements $\Lambda_{ii}^{-1} = 1/\lambda_i$
- This is only defined if $\mathbf{M}$ if all the eigenvalues of $\mathbf{M}$ are non-zero ($\mathbf{M}$ is said to be full rank)
- If λ_i is very small then $1/\lambda_i$ is large and in taking the inverse $\mathbf{M}^{-1}\mathbf{x}$ any component of $\mathbf{x}$ in the direction $\mathbf{v}_i$ will get magnified by $1/\lambda_i$
- For linear regression we invert $\mathbf{M} = \mathbf{X}^T \mathbf{X}$
 - in directions where the training examples don't vary much the associated eigenvalue will be small and the inverse inherently unstable

3 Exercises

3.1 Linear Regression

- Derive the formula for the weight vector in linear regression

4 Experiments

4.1 Eigensystems

- In either Matlab/Octave or python generate random matrices and check the matrix identities

```
X = randn(5,4) % generate a mock designer matrix with 5 inputs of length 4
M = X'*X       % compute a symmetrix matrix
[V,L] = eig(M) % compute eigenvalues
V*L*V'         % should be identical to M
V*V'           % should be the identity matrix (up to rounding precision)
V'*V           % should be the identity matrix (up to rounding precision)
x = randn(4,1) % generate a random column matrix of length 4
y = randn(4,1) % generate another random column matrix of length 4
xp = V*x       % apply V to x
yp = V*y       % apply V to y
norm(x)        % compute Euclidean norm of x
norm(xp)       % should be the same as Euclidean nor of xp
x'*y           % compute inner product of x and y
xp'*yp'        % compute inner produce of xp and yp (should be the same as above)

Z = rand(4,5)  % consider a designer matrix where we would have more unknowns the examples
W = Z'*Z       % compute a covariance type matrix (except we don't subtract the mean
eig(W)         % compute eigenvalues (one should be 0 up to machine precision)
```